

# Practical Report: Analysing a Simple HTTP Request Using Wireshark

---

## Part A – Theoretical Understanding

### Q1. What is Wireshark? Explain its key features and use cases.

Wireshark is a widely used **open-source network protocol analyzer** that enables real-time or offline packet inspection. It allows users to **capture, view, filter, and analyze network traffic at the packet level**, making it an essential tool for networking professionals, developers, and cybersecurity analysts.

#### Key Features:

1. **Real-time packet capture & offline analysis** – Supports live monitoring of network traffic as well as loading pre-saved capture files.
2. **Extensive protocol support** – Recognizes hundreds of protocols, from low-level (Ethernet, IP, TCP/UDP) to application-level (HTTP, DNS, FTP, etc.).
3. **Detailed packet inspection** – Presents a hierarchical, decoded view of each packet, allowing in-depth exploration of headers and payloads.
4. **Advanced filtering** – Provides both **capture filters** (before capture) and **display filters** (after capture) to focus on relevant traffic.
5. **Statistical analysis tools** – Includes protocol hierarchies, conversation tracking, and flow graphs.
6. **Cross-platform availability** – Compatible with Windows, macOS, and Linux.

#### Common Use Cases:

- **Network troubleshooting** – Diagnosing latency, packet loss, or protocol errors.
  - **Security analysis** – Detecting suspicious traffic patterns, intrusion attempts, or data exfiltration.
  - **Protocol development & learning** – Observing and debugging protocol implementations in real time.
-

## Q2. Types of Filters in Wireshark

Wireshark supports two main types of filters:

### 1. Capture Filters

- Applied **before** packet capture.
- Determines which packets Wireshark stores.
- Uses **Berkeley Packet Filter (BPF)** syntax.  
**Examples:**
  - `host 192.168.1.1` → Captures only packets to/from IP `192.168.1.1`
  - `port 80` → Captures only packets using port 80 (HTTP traffic)

### 2. Display Filters

- Applied **after** capture.
- Controls which packets are **shown** in the analysis window.
- Uses Wireshark's **own filter syntax** (case-sensitive).  
**Examples:**
  - `http` → Shows only HTTP protocol packets.
  - `ip.src == 10.0.0.5 && tcp.port == 443` → Shows packets from `10.0.0.5` using TCP port 443 (HTTPS traffic).

---

## Q3. Wireshark in Cybersecurity: Role in Threat Detection & Analysis

Wireshark is a vital tool for **network forensics and security analysis** because it can inspect and decode packets at a granular level.

### Applications in Cybersecurity:

1. **Threat detection** – Identifies anomalies such as unusual IP addresses, suspicious protocols, repeated login failures, or malicious payloads.
2. **Incident forensics** – Allows analysts to trace attacks, reconstruct timelines, and collect evidence (e.g., stolen data, attacker IPs).
3. **Malware investigation** – Monitors malicious command-and-control (C2) traffic and detects potential data leaks.
4. **Policy compliance** – Ensures sensitive data (like passwords) is not sent in plaintext.
5. **Rogue device detection** – Flags unauthorized hosts or fake DNS/DHCP servers.

With **filtering, detailed protocol dissection, and historical analysis**, Wireshark supports both **proactive monitoring** and **post-incident investigations**.

---

## Part B – Practical Implementation

### Objective:

- Understand HTTP request/response structure.
  - Identify source & destination IP addresses.
  - Extract key request/response metadata.
- 

### Tools Used

- **Wireshark** – Network protocol analyzer.
  - **Web Browser** – Used to generate HTTP requests (<http://example.com>).
- 

### Procedure

#### 1. Start Packet Capture

1. Open Wireshark.
2. Select the active **network interface** (Wi-Fi or Ethernet).
3. Click **Start Capturing**.

#### 2. Generate HTTP Traffic

1. Open a browser.
2. Visit a **non-HTTPS** site (e.g., <http://example.com>) to ensure traffic is **unencrypted**.

#### 3. Stop Capture

- Once the page loads, click **Stop** in Wireshark.

#### 4. Apply Display Filter

- Enter [http](http://example.com) in the **Display Filter** bar to show only HTTP packets.

---

## Analysis

### 1. Identifying the HTTP Request

- Look for a **GET** or **POST** request in the **Info** column.  
*Example: GET / HTTP/1.1*

### 2. Source & Destination IP Addresses

- Select the HTTP request packet.
- Expand **Internet Protocol Version 4 (IPv4)** in the details pane.  
*Example:*
  - **Source IP:** 192.168.1.5 (local machine)
  - **Destination IP:** 93.184.216.34 (example.com server)

### 3. Extracting HTTP Request Information

- Expand **Hypertext Transfer Protocol** in the details pane.
  - **Host:** example.com
  - **User-Agent:** Mozilla/5.0 (Windows NT 10.0; Win64; x64)...
  - **Accept:** text/html,application/xhtml+xml,...

### 4. Analyzing the HTTP Response

- Locate the matching **HTTP/1.1 200 OK** packet.
- Expand **Hypertext Transfer Protocol** to view:
  - **Status Code:** 200 (Success)
  - **Content-Type:** text/html
  - **Server:** ECS (example server)
  - **Content-Length:** *size of HTML content*

---

## Key Observations

- HTTP traffic is **human-readable** in plaintext (when not encrypted with HTTPS).

- Packet headers reveal **browser type**, **server software**, and **resource size**.
  - IP addresses and HTTP headers can be used for **basic network mapping** and **traffic profiling**.
- 

## Conclusion

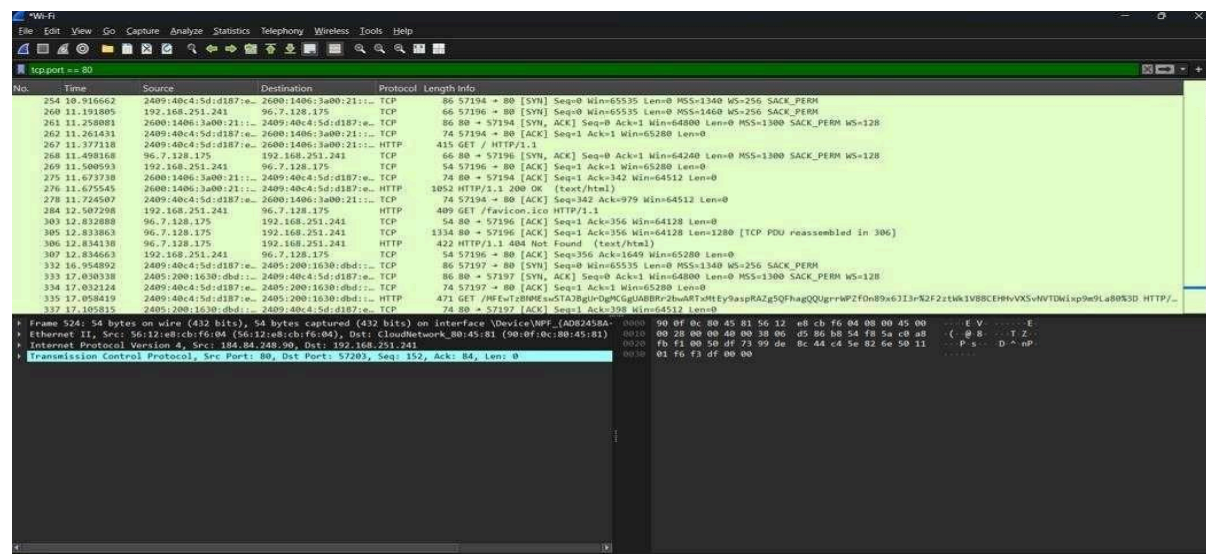
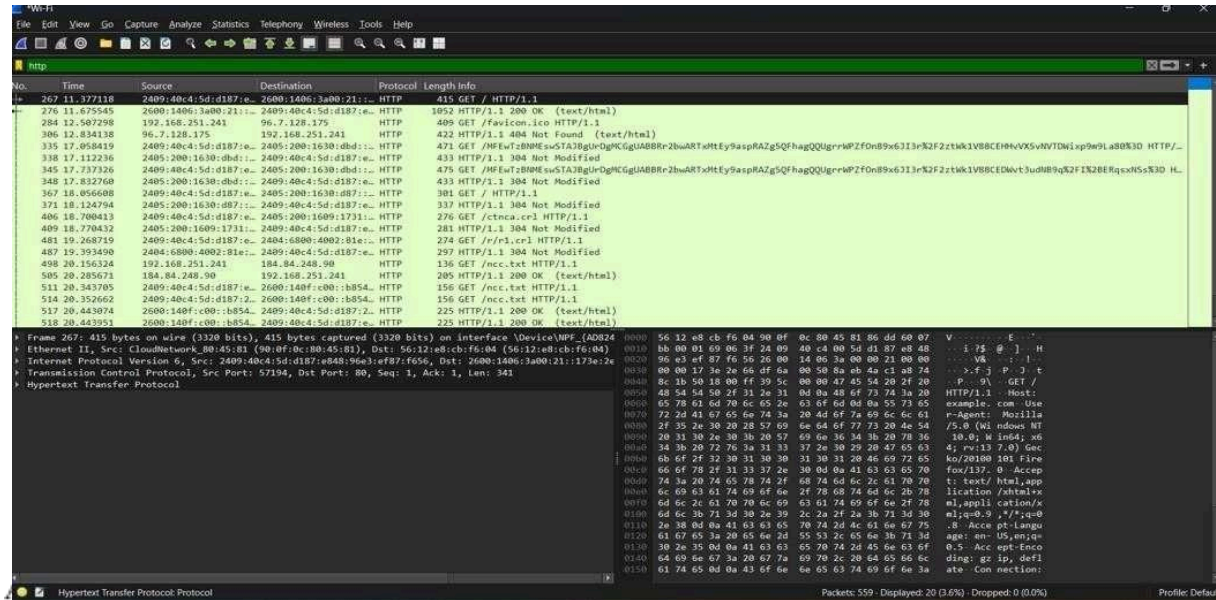
This exercise demonstrated how Wireshark can be used to:

- Capture and filter HTTP traffic.
- Identify source/destination network endpoints.
- Extract key HTTP request and response details.

Such analysis is fundamental in **network troubleshooting**, **web debugging**, and **cybersecurity monitoring**.

# Screenshots

## 1. Start Capturing Packets



## TCP Analysis

Total packets exchanged during the handshake process: 3 packets  
Flags used in each packet:

1. First packet – SYN
2. Second packet – SYN-ACK
3. Third packet – ACK

Count of SYN packets observed: 18 SYN packets were sent in total

The screenshot shows a Wireshark interface with a packet capture titled "assignmentEpcap". The filter bar at the top displays "tcp.flags.syn == 1 && tcp.flags.ack == 0". The main pane shows a list of packets with columns for No., Time, Source, Destination, Protocol, Length, and Info.

| No. | Time      | Source              | Destination          | Protocol | Length | Info                                        |
|-----|-----------|---------------------|----------------------|----------|--------|---------------------------------------------|
| 14  | 0.570597  | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 37  | 3.189866  | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 132 | 6.397046  | 192.168.251.241     | 137.76.187.104       | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 195 | 9.054435  | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 254 | 10.916662 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 254 | 30.917776 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 260 | 11.191805 | 192.168.251.241     | 96.7.128.175         | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 332 | 16.954992 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 349 | 17.306640 | 192.168.251.241     | 96.7.128.175         | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 358 | 17.942820 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 383 | 16.493916 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 399 | 18.502113 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 479 | 19.143742 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 495 | 20.008587 | 192.168.251.241     | 184.84.248.90        | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 506 | 20.286942 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 507 | 20.297117 | 2409:40c4:5d:d187:: | 2060:140f:c00:b054:: | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 553 | 22.586894 | 192.168.251.241     | 13.68.233.9          | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |
| 554 | 22.737827 | 192.168.251.241     | 13.68.233.9          | TCP      |        | Seq=0 Win=0 Len=0 MSS=1340 WS=256 SACK_PERM |

[illegible]

## **Conclusion**

Using Wireshark, we successfully captured and analysed a simple HTTP GET request and response. The analysis helped us:

- Identify the source (client) and destination (server) IP addresses.
- Extract useful header information like Host and User-Agent.
- Understand the HTTP communication process between client and server.